Department of Computer Science and Engineering

24X0598: PARALLEL COMPUTATION: RUST

LAB MANUAL

1	**Department**	COMPUTER SCIENCE AND ENGINEERING							
2	**Course Name**	24X0598: PARALLEL COMPUTATION: RUST							
3	**Course Code**	24X0598							
4	**Year/Semester**	II/II							
5	**Regulation**	MLRS-R24							
6	**Structure of the course**	Theory				Practical			
		Lecture 3	Tutorials 0	Practical 0	Credit 3	L 0	T 0	P 0	C 1
7	**Type of course**	BS ×	HS ×	ES ×	PC x	PE ×	OE ×	CC ×	MC ×
8	**Course Offered**	Odd Semester	X	Even Semester	✓				
9	Total lecture, tutorial and practical hours for this course Offered (16 weeks of teaching per semester)								
	Lectures: 48 Hours	**Tutorials:** 0 hours	**Practical:** 0 hours						
10	**Course Coordinator**	Dr. S. Pratap Singh							
11	**Date Approved by BOS**								
12	**Course Webpage**	www.mlritm.ac.in/							
13	**Prerequisites/Co-requisites**	Level	Course Code	Semester	Prerequisites				
		-		II,	Basics of any programming language				

Department of Computer Science and Engineering

Vision of the Department

- To empower the students to be technologically adept, innovative, self-motivated and responsible global citizen possessing human values and contribute significantly towards high quality technical education by harmonizing innovation with sustainability.

Mission of the Department

- M1: To offer high-quality education in the computing fields by providing an environment where the knowledge is gained and applied to participate in research, for both students and faculty.
- M2: To develop the problem solving skills in the students to be ready to deal with cutting edge technologies of the industry.
- M3: To make the students and faculty excel in their professional fields by inculcating the communication skills, leadership skills, team building skills with the organization of various co-curricular and extra-curricular programmes.
- M4: To provide the students with theoretical and applied knowledge, and adopt an education approach that promotes lifelong learning and ethical growth.

Program Educational Objectives (PEOs)

- PEO 1: To induce strong foundation in mathematical and core concepts, which enable them to participate in research, in the field of computer science.
- PEO 2: To be able to become the part of application development and sustainability development by learning the computer programming methods, of the industry and related domains.
- PEO 3: To Gain the multidisciplinary knowledge by understanding the scope of association of computer science engineering discipline with other engineering disciplines.
- PEO 4: To improve soft skills which build the professional qualities, there by understanding the social responsibilities and ethical attitude.

Program Outcomes (POs)

- **PO1 ENGINEERING KNOWLEDGE:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

- **PO2 PROBLEM ANALYSIS:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

- **PO3 DESIGN/DEVELOPMENT OF SOLUTIONS:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

- **PO4 CONDUCT INVESTIGATIONS OF COMPLEX PROBLEMS:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

- **PO5 MODERN TOOL USAGE:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

- **PO6 THE ENGINEER AND SOCIETY:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

- **PO7 ENVIRONMENT AND SUSTAINABILITY:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

- PO8 ETHICS: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

- **PO9 INDIVIDUAL AND TEAM WORK:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

- **PO10 COMMUNICATION:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, give and receive clear instructions.

- **PO11 PROJECT MANAGEMENT AND FINANCE:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

- **PO12 LIFE-LONG LEARNING:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

- **Applications of Computing:** Ability to use knowledge in various domains to provide solution to new ideas and innovations.
- **Programming Skills:** Identify required data structures, design suitable algorithms, develop and maintain software for real world problems.
- **Entrepreneur and higher studies:** Make use of computational and experimental knowledge for creating innovative career paths, to be an
- knowledge for creating innovative career paths, to be an entrepreneur and desire for higher studies

MLRITM

RUST PROGRAMMING LAB MANUAL

24X0598: PARALLEL COMPUTATION: RUST

L	T	P	C
0	0	2	1

Module I – Basics of Rust

1. **Hello, World!**
   - Basic Rust program structure using fn main().
2. **Variable Declaration and Mutability**
   - Using let, mut, and type inference.
3. **Data Types Demo**
   - Use i32, f64, char, bool, etc., in a single program.
4. **Arithmetic and Logical Operators**
   - Calculator-style program showing arithmetic and logical operations.
5. **Using cargo for a Simple Project**
   - Create a simple project using cargo new, build, and run it.

Module II – Ownership and Control Flow

6. **Ownership Transfer**
   - Pass variables to functions and explore ownership and move.
7. **Borrowing and References**
   - Demonstrate &T and &mut T references and restrictions.
8. **Lifetimes Example**
   - Use lifetime annotations in a function returning references.
9. **If-Else Statement**
   - Even/odd checker or grade calculator.
10. **Looping Examples**
    - Use for, while, and loop to iterate over arrays or counters.
11. **Pattern Matching with match**
    - Match numbers to print weekdays or match enums.
12. **Nested Control Statements**
    - Combine loops and conditionals in one program (e.g., a simple number guessing game).

Module III – Functions and Structs

13. **Functions with Arguments and Return Values**
    - Create a function to calculate factorial or square of a number.
14. **Ownership in Functions**
    - Pass by value vs. reference in function parameters.
15. **Using Structs**
    - Define a Rectangle struct and calculate area.
16. **Tuple Structs and Field Init Shorthand**
    - Define a Color(u8, u8, u8) and use field shorthand for init.

17. **Enums and Pattern Matching**
    - Define an enum for TrafficLight and match its values.
18. **Option Enum Usage**
    - Safe division function returning Option<f64>.

Module IV – Project Management and Smart Pointers

19. **Error Handling with Result**
    - File reading or division with error handling using Result.
20. **Using Box, Rc, and RefCell**
    - Demonstrate smart pointers with a simple linked list or counter program.

Module- V – Object oriented Programming

21. **"Polymorphic Behavior with Trait Objects: A Speakable Animal Zoo"**
Covers: Traits, dynamic dispatch, trait objects (&dyn Trait)

22. **"Implementing Strategy Pattern with Traits for Payment Processing"**
Covers: Object-oriented design using traits and Box<dyn Trait> for dynamic strategy switching

23. **"Trait-Based Drawing Application with Heterogeneous UI Components"**
Covers: Object-oriented characteristics, trait objects, and allowing different drawable types in a single collection

Introduction to RUST Programming:

Rust is a modern systems programming language developed by the Mozilla Corporation. It is intended to be a language for highly concurrent and highly secure systems. It compiles to native code; hence, it is blazingly fast like C and C++. This tutorial adopts a simple and practical approach to describe the concepts of Rust programming.

Rust is a systems level programming language, developed by Graydon Hoare. Mozilla Labs later acquired the programme.

Application v/s Systems Programming Languages

Application programming languages like Java/C# are used to build software, which provide services to the user directly. They help us build business applications like spreadsheets, word processors, web applications or mobile applications.

Systems programming languages like C/C++ are used to build software and software platforms. They can be used to build operating systems, game engines, compilers, etc. These programming languages require a great degree of hardware interaction.

Systems and application programming languages face two major problems −

- It is difficult to write secure code.
- It is difficult to write multi-threaded code.

Why Rust?

Rust focuses on three goals −

- Safety
- Speed
- Concurrency

The language was designed for developing highly reliable and fast software in a simple way. Rust can be used to write high-level programs down to hardware-specific programs.

Performance

Rust programming language does not have a Garbage Collector (GC) by design. This improves the performance at runtime.

Memory safety at compile time

Software built using Rust is safe from memory issues like dangling pointers, buffer overruns and memory leaks.

Multi-threaded applications

Rusts ownership and memory safety rules provide concurrency without data races.

Support for Web Assembly (WASM)

Web Assembly helps to execute high computation intensive algorithms in the browser, on embedded devices, or anywhere else. It runs at the speed of native code. Rust can be compiled to Web Assembly for fast, reliable execution.

Rust - Environment Setup

Installation of Rust is made easy through **rustup**, a console-based tool for managing Rust versions and associated tools.

Installation on Windows

Let us learn how to install RUST on Windows.

- Installation of Visual Studio 2013 or higher with C++ tools is mandatory to run the Rust program on windows. First, download Visual Studio from here VS 2013 Express
- Download and install **rustup** tool for windows. **rustup-init.exe** is available for download here – Rust Lang
- Double-click **rustup-init.exe** file. Upon clicking, the following screen will appear.

```
C:\Users\dell\Downloads\rustup-init.exe

Welcome to Rust!

This will download and install the official compiler for the Rust programming
language, and its package manager, Cargo.

It will add the cargo, rustc, rustup and other commands to Cargo's bin
directory, located at:

  C:\Users\dell\.cargo\bin

This path will then be added to your PATH environment variable by modifying the
HKEY_CURRENT_USER/Environment/PATH registry key.

You can uninstall at any time with rustup self uninstall and these changes will
be reverted.

Current installation options:

   default host triple: x86_64-pc-windows-msvc
     default toolchain: stable
  modify PATH variable: yes

1) Proceed with installation (default)
2) Customize installation
3) Cancel installation
>
```

- press *enter* for default installation. Once installation is completed, the following screen appears.

```
Rust is installed now. Great!

To get started you need Cargo's bin directory (%USERPROFILE%\.cargo\bin) in
your PATH environment variable. Future applications will automatically have the
correct environment, but you may need to restart your current shell.

Press the Enter key to continue.
```

- From the installation screen, it is clear that Rust related files are stored in the folder – C:\Users\{PC}\.cargo\bin

The contents **of the folder are –**

cargo-fmt.exe
cargo.exe
rls.exe
rust-gdb.exe
rust-lldb.exe
rustc.exe // this is the compiler for rust
rustdoc.exe
rustfmt.exe
rustup.exe

- **Cargo** is the package manager for Rust. To verify if **cargo** is installed, execute the following command –
  C:\Users\Admin>cargo -V

cargo 1.29.0 (524a578d7 2018-08-05)

Administrator: Command Prompt

```
               5 Dir(s)  139,817,762,816 bytes free

C:\Users\Administrator\Desktop\pratap>cargo -V
cargo 1.86.0 (adf9b6ad1 2025-02-28)

C:\Users\Administrator\Desktop\pratap>
```

The basic syntax of Rust language through a **HelloWorld** example.

- Create a **HelloWorld-App** folder and navigate to that folder on terminal

C:\Users\Admin>mkdir HelloWorld-App
C:\Users\Admin>cd HelloWorld-App
C:\Users\Admin\HelloWorld-App>

- To create a Rust file, execute the following command −

C:\Users\Admin\HelloWorld-App>notepad Hello.rs

- Rust program files have an extension .rs.

The above command creates an empty file **Hello.rs** and opens it in NOTEpad.

List Of Module

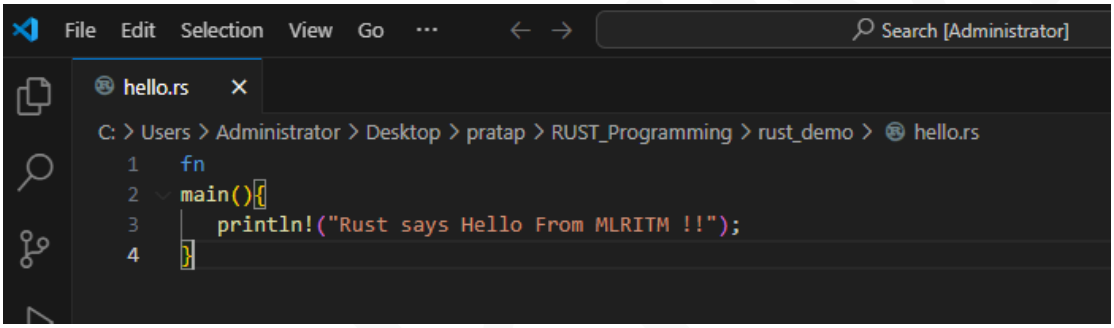
Module I – Basics of Rust

1. **Hello, World!**

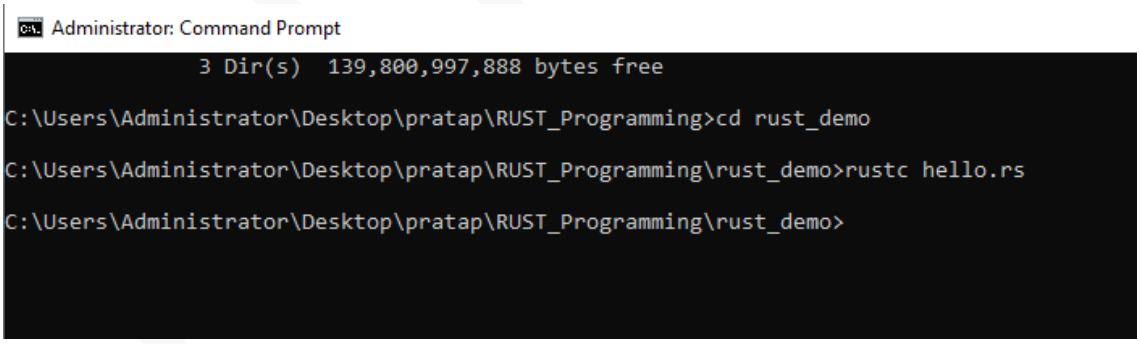
Basic Rust program structure using fn main().

```
fn main()
{
println!("Rust says Hello from MLRITM !!");
 }
```

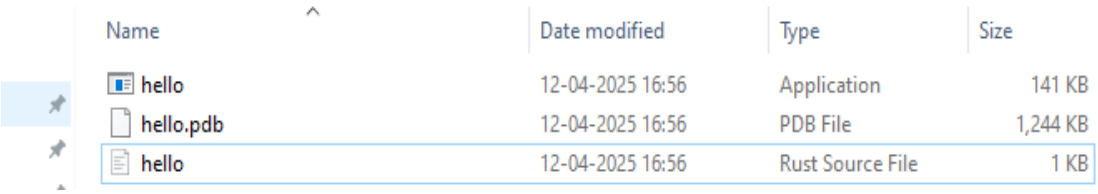
- Write the program and save it with **filename.rs** where filename is name of the file and rs is the extension of the file.



- Compile the hello.rs file with **rustc filename.rs**



- After compilation the following files will be created.



Name	Date modified	Type	Size
hello	12-04-2025 16:56	Application	141 KB
hello.pdb	12-04-2025 16:56	PDB File	1,244 KB
hello	12-04-2025 16:56	Rust Source File	1 KB

- To see the output of the file use **filename.exe** in the command prompt.

```
Administrator: Command Prompt

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>hello.exe
Rust says Hello From MLRITM !!

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

2. **Variable Declaration and Mutability**
   - Using let, mut, and type inference.

Syntax

The data type is optional while declaring a variable in Rust. The data type is inferred from the value assigned to the variable.

The syntax for declaring a variable is given below.

```
let variable_name = value;          // no type specified

let variable_name:dataType = value;   //type specified
```

example program :

```
fn main()
{
let fees = 25_000;
let salary:f64 = 35_000.00;
println!("fees is {} and salary is {}",fees,salary);
}
```

Output:

```
Administrator: Command Prompt
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>notepad var.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc var.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>var.exe
fees is 25000 and salary is 35000

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>_
```

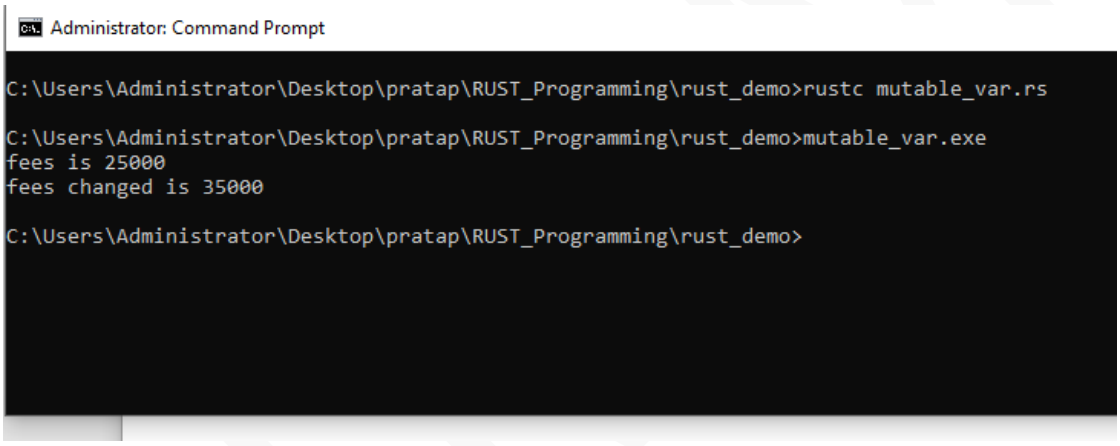
Mutable

Variables are immutable by default. Prefix the variable name with mut keyword to make it mutable. The value of a mutable variable can be changed.

Example program:

```
fn main() {
   let mut fees:i32 = 25_000;
   println!("fees is {} ",fees);
   fees = 35_000;
   println!("fees changed is {}",fees);
}
```

Output:



Immutable

By default, variables are immutable − read only in Rust. In other words, the variable's value cannot be changed once a value is bound to a variable name.

Let us understand this with an example.

```
fn main() {
   let fees = 25_000;
   println!("fees is {} ",fees);
   fees = 35_000;
   println!("fees changed is {}",fees);
}
```

The output will be as shown below −

```
Administrator: Command Prompt
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>notepad immutable_var.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc immutable_var.rs
error[E0384]: cannot assign twice to immutable variable `fees`
 --> immutable_var.rs:4:4
  |
2 |     let fees = 25_000;
  |         ---- first assignment to `fees`
3 |     println!("fees is {} ",fees);
4 |     fees = 35_000;
  |     ^^^^^^^^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
2 |     let mut fees = 25_000;
  |         +++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0384`.

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

3. **Data Types Demo**
   - Use i32, f64, char, bool, etc., in a single program.

Program:

```
fn main()
{
  let company_string = "Marri Laxman Reddy Institute of Technology and
Management";                          // string type
  let rating_float = 14.5;            // float type
  let is_growing_boolean = true;      // boolean type
  let icon_char = ' ';                //unicode character type
  println!("company name is:{}",company_string);
  println!("company rating on 5 is:{}",rating_float);
  println!("company is growing :{}",is_growing_boolean);
  println!("company icon is:{}",icon_char);

}
```

```rust
fn main()
{
    let company_string = "Marri Laxman Reddy Institute of Technology and Management";  // string type
    let rating_float = 14.5;                // float type
    let is_growing_boolean = true;          // boolean type
    let icon_char = ' ';                    //unicode character type

    println!("company name is:{}",company_string);
    println!("company rating on 5 is:{}",rating_float);
    println!("company is growing :{}",is_growing_boolean);
    println!("company icon is:{}",icon_char);
}
```

OUTPUT:

```
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc variable_demo.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>variable_demo.exe
company name is:Marri Laxman Reddy Institute of Technology and Management
company rating on 5 is:14.5
company is growing :true
company icon is:

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Scalar Types

A scalar type represents a single value. For example, 10,3.14,'c'. Rust has four primary scalar types.

- Integer
- Floating-point
- Booleans
- Characters

Example program:

scalar_datatype - Notepad

File Edit Format View Help

```
fn main() {
   let result = 10;    // i32 by default
   let age:u32 = 20;
   let sum:i32 = 5-15;
   let mark:isize = 10;
   let count:usize = 30;
   println!("result value is {}",result);
   println!("sum is {} and age is {}",sum,age);
   println!("mark is {} and count is {}",mark,count);
}
```

Output:

```
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>notepad scalar_datatype.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc scalar_datatype.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>scalar_datatype.exe
result value is 10
sum is -10 and age is 20
mark is 10 and count is 30

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Float datatype : example program:

```
fn main()
{
  let result = 10.00;       //f64 by default
  let interest:f32 = 8.35;
  let cost:f64 = 15000.600;  //double precision

  println!("result value is {}",result);
  println!("interest is {}",interest);
  println!("cost is {}",cost);
}
```

Output:

Administrator: Command Prompt

```
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc float_datatype.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>float_datatype.exe
result value is 10
interest is 8.35
cost is 15000.6

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Automatic Type Casting

Automatic type casting is not allowed in Rust. Consider the following code snippet. An integer value is assigned to the float variable ***interest***.

```
fn main()
{

 let interest:f32 = 8; // integer assigned to float variable
println!("interest is {}",interest);

}
```

Select Administrator: Command Prompt

```
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>notepad typecase_var.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc typecase_var.rs
error[E0308]: mismatched types
 --> typecase_var.rs:2:23
  |
2 |     let interest:f32 = 8;   // integer assigned to float variable
  |                 ---    ^ expected `f32`, found integer
  |                 |
  |                 expected due to this
  |
help: use a float literal
  |
2 |     let interest:f32 = 8.0;   // integer assigned to float variable
  |                         ++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0308`.

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

4. **Arithmetic and Logical Operators**
   - Calculator-style program showing arithmetic and logical operations.

Steps for creating calculator program in rust:

1. Create new project in rust using : cargo new rust_calculator
2. Open the new project in visual code and open main.rs file.
3. Write the calculator program in main,rs file and save it.
4. Run the project using the command : cargo run.

```
Administrator: C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>cargo new rust_calculator
    Creating binary (application) `rust_calculator` package
note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html
```

```
Administrator: C:\Windows\System32\cmd.exe

 Directory of C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator

13-04-2025  10:08    <DIR>          .
13-04-2025  10:08    <DIR>          ..
13-04-2025  10:08                86 Cargo.toml
13-04-2025  10:08    <DIR>          src
               1 File(s)             86 bytes
               3 Dir(s)  139,668,422,656 bytes free
```

```
use std::io;

fn main() {
    let mut x: String = String::new();
    let mut y: String = String::new();
    let result: i32;
    let mut op: String = String::new();

    println!("Enter the first number: ");
    io::stdin().read_line(&mut x).expect("Invalid Input");
    let x: i32 = match x.trim().parse() {
        Ok(num) => num,
        Err(_) => {
            println!("Invalid input!");
            return;
        }
    };

    println!("Enter the second number: ");
    io::stdin().read_line(&mut y).expect("Invalid Input");
    let y: i32 = match y.trim().parse() {
        Ok(num) => num,
        Err(_) => {
            println!("Invalid input!");
            return;
        }
    };

    println!("List of operators:");
    println!("(1) Add");
    println!("(2) Subtract");
    println!("(3) Multiply");
    println!("(4) Divide");
    println!("Select the number associated with the desired operation: ");

    io::stdin().read_line(&mut op).expect("Invalid Input");
```

Program:

```
use std::io;
fn main() {
    let mut x: String = String::new();
    let mut y: String = String::new();
    let result: i32;
    let mut op: String = String::new();

    println!("Enter the first number: ");
    io::stdin().read_line(&mut x).expect("Invalid Input");
```

```
    let x: i32 = match x.trim().parse() {
        Ok(num) => num,
        Err(_) => {
            println!("Invalid input!");
            return;
        }
    };

    println!("Enter the second number: ");
    io::stdin().read_line(&mut y).expect("Invalid Input");
    let y: i32 = match y.trim().parse() {
        Ok(num) => num,
        Err(_) => {
            println!("Invalid input!");
            return;
        }
    };

    println!("List of operators:");
    println!("(1) Add");
    println!("(2) Subtract");
    println!("(3) Multiply");
    println!("(4) Divide");
    println!("Select the number associated with the desired operation: ");

    io::stdin().read_line(&mut op).expect("Invalid Input");
    let op: i32 = match op.trim().parse() {
        Ok(num) => num,
        Err(_) => {
            println!("Invalid input!");
            return;
        }
    };

    match op {
        1 => result = x + y,
        2 => result = x - y,
        3 => result = x * y,
        4 => result = x / y,
        _ => {
            println!("Invalid selection");
            return;
        }
    }

    println!("The result is: {}", result);
}
```

OUTPUT:

```
Administrator: C:\Windows\System32\cmd.exe

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator>cargo run
   Compiling rust_calculator v0.1.0 (C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.75s
     Running `target\debug\rust_calculator.exe`
Enter the first number:
10
Enter the second number:
20
List of operators:
(1) Add
(2) Subtract
(3) Multiply
(4) Divide
Select the number associated with the desired operation:
1
The result is: 30
```

```
Administrator: C:\Windows\System32\cmd.exe
operable program or batch file.

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator>cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target\debug\rust_calculator.exe`
Enter the first number:
10
Enter the second number:
20
List of operators:
(1) Add
(2) Subtract
(3) Multiply
(4) Divide
Select the number associated with the desired operation:
2
The result is: -10

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator>cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target\debug\rust_calculator.exe`
Enter the first number:
10
Enter the second number:
20
List of operators:
(1) Add
(2) Subtract
(3) Multiply
(4) Divide
Select the number associated with the desired operation:
3
The result is: 200

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator>cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
     Running `target\debug\rust_calculator.exe`
Enter the first number:
10
Enter the second number:
20
List of operators:
(1) Add
(2) Subtract
(3) Multiply
(4) Divide
Select the number associated with the desired operation:
4
The result is: 0
```

b) Logical operator program

logical_operator - Notepad

File Edit Format View Help

```
fn main() {
    let a = 20;
    let b = 30;

    if (a > 10) && (b > 10) {
        println!("true");
    }
    let c = 0;
    let d = 30;

    if (c>10) || (d>10){
        println!("true");
    }
    let is_elder = false;

    if !is_elder {
        println!("Not Elder");
    }
}
```

Ln 19 100% Windows (CRLF) UTF-8

OUTPUT:

Administrator: C:\Windows\System32\cmd.exe

```
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\rust_calculator>cd ..

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>notepad logical_operator.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc logical_operator.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>logical_operator.exe
true
true
Not Elder

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

5. **Using cargo for a Simple Project**

   Create a simple project using cargo new, build, and run it.

Creating a Project with Cargo

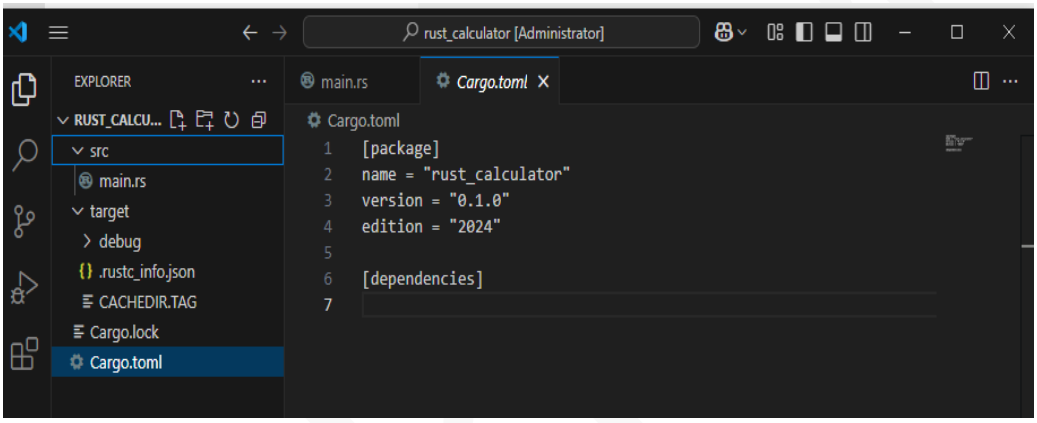
Let's create a new project using Cargo and look at how it differs from our original "Hello, world!" project. Navigate back to your *projects* directory (or wherever you decided to store your code). Then, on any operating system, run the following:

$ cargo new hello_cargo

$ cd hello_cargo

The first command creates a new directory and project called *hello_cargo*. We've named our project *hello_cargo*, and Cargo creates its files in a directory of the same name.
Go into the *hello_cargo* directory and list the files. You'll see that Cargo has generated two files and one directory for us: a *Cargo.toml* file and a *src* directory with a *main.rs* file inside.
It has also initialized a new Git repository along with a *.gitignore* file. Git files won't be generated if you run cargo new within an existing Git repository; you can override this behavior by using cargo new --vcs=git.

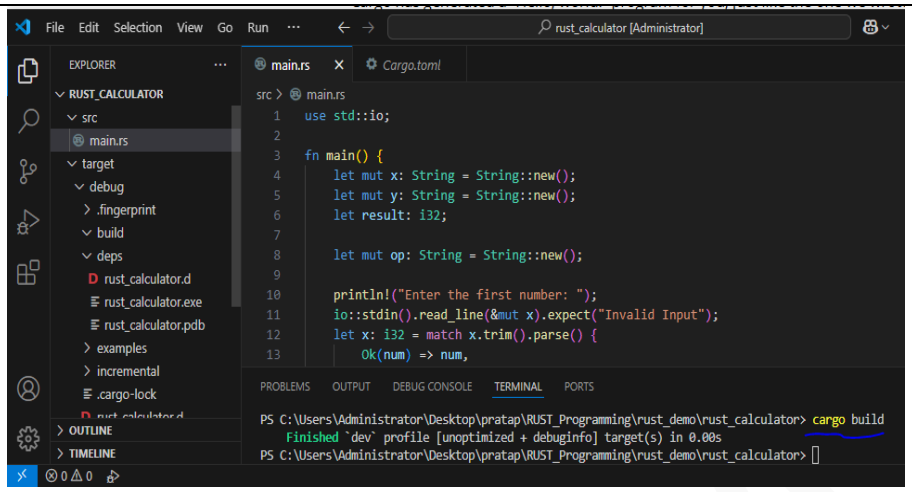
Open *Cargo.toml* in your text editor of choice. It should look similar to the code.



Now open *src/main.rs* and take a look:
Filename: src/main.rs

```
fn main() {
    println!("Hello, world!");
}
```

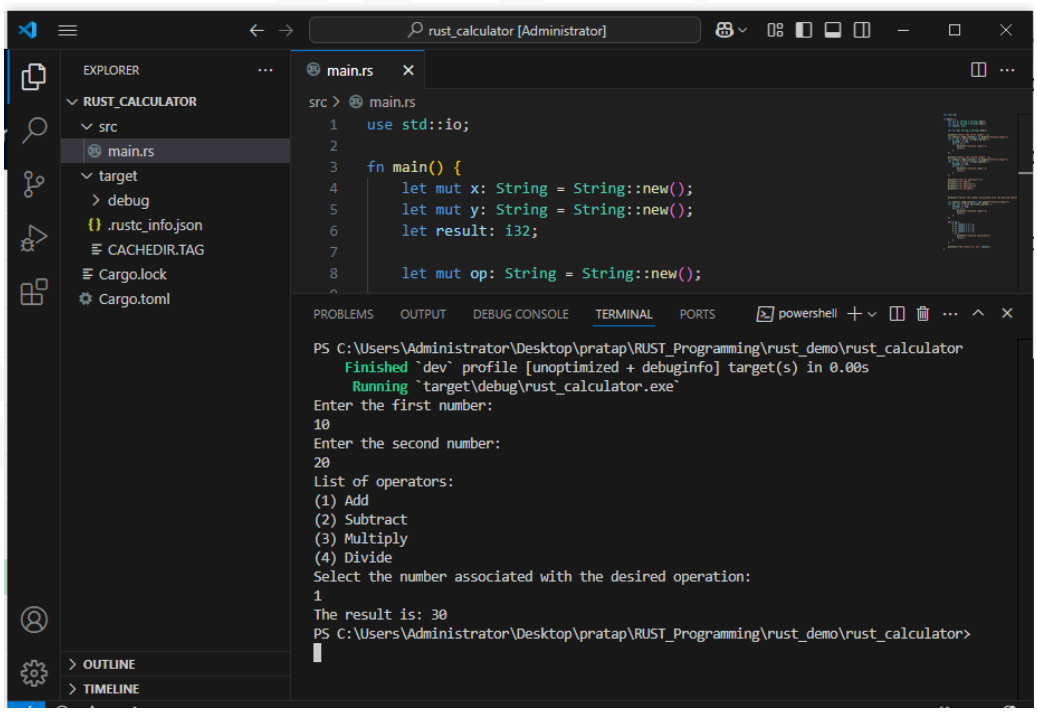
Cargo has generated a "Hello, world!" program for you, just like the one we wrote in Listing 1-1! So far, the differences between our project and the project Cargo generated are that Cargo placed the code in the *src* directory and we have a *Cargo.toml* configuration file in the top directory.



Executing The project:

$ cargo run
Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
Running `target/debug/hello_cargo`
Hello, world!

Note : The example screenshot for rust_calculator program taken.



Module II – Ownership and Control Flow

6. **Ownership Transfer**

- Pass variables to functions and explore ownership and move.

The memory for a program can be allocated in the following −

- Stack
- Heap

What is Ownership?

Each value in Rust has a variable that is called **owner** of the value. Every data stored in Rust will have an owner associated with it. For example, in the syntax − *let age = 30, age* is the owner of the value *30*.

- Each data can have only one owner at a time.
- Two variables cannot point to the same memory location. The variables will always be pointing to different memory locations.

Transferring Ownership

The ownership of value can be transferred by −

- Assigning value of one variable to another variable.
- Passing value to a function.
- Returning value from a function.

Assigning value of one variable to another variable

The key selling point of Rust as a language is its memory safety. Memory safety is achieved by tight control on who can use what and when restrictions.

Program:

```
Scenario 1:


fn main(){
   let v = vec![1,2,3];     // vector v owns the object in heap
   let v2 = v;              // moves ownership to v2
   display(v2);             // v2 is moved to display and v2 is invalidated
   println!("In main {:?}",v2);    //v2 is No longer usable here
}
fn display(v:Vec<i32>){
   println!("inside display {:?}",v);
}
```

Output:

```
Administrator: C:\Windows\System32\cmd.exe
Not Elder

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>notepad passing_value_function.rs

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>rustc passing_value_function.rs
error[E0382]: borrow of moved value: `v2`
 --> passing_value_function.rs:5:28
  |
3 |     let v2 = v;              // moves ownership to v2
  |         -- move occurs because `v2` has type `Vec<i32>`, which does not implement the `Copy` trait
4 |     display(v2);             // v2 is moved to display and v2 is invalidated
  |             -- value moved here
5 |     println!("In main {:?}",v2);    //v2 is No longer usable here
  |                             ^^ value borrowed here after move
  |
note: consider changing this parameter type in function `display` to borrow instead if owning the value isn't necessary
 --> passing_value_function.rs:7:14
  |
7 | fn display(v:Vec<i32>){
  |    -------   ^^^^^^^^ this parameter takes ownership of the value
  |    |
  |    in this function
  = note: this error originates in the macro `$crate::format_args_nl` which comes from the expansion of the macro `print
ln` (in Nightly builds, run with -Z macro-backtrace for more info)
help: consider cloning the value if the performance cost is acceptable
  |
4 |     display(v2.clone());             // v2 is moved to display and v2 is invalidated
  |               ++++++++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0382`.

C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Scenario 2:

```
fn main(){
   let u1 = 10;
   let u2 = u1;  // u1 value copied(not moved) to u2

   println!("u1 = {}",u1);
  // println!("u2 = {}",u2);
}
```

```
warning: unused variable: `u2`
 --> main.rs:3:8
  |
3 |     let u2 = u1;  // u1 value copied(not moved) to u2
  |         ^^ help: if this is intentional, prefix it with an underscore: `_u2`
  |
  = note: `#[warn(unused_variables)]` on by default

warning: 1 warning emitted

u1 = 10
```

Scenario 3:

```
fn main(){
   let u1 = 10;
   let u2 = u1;  // u1 value copied(not moved) to u2


   println!("u1 = {}",u1);
   println!("u2 = {}",u2);
```

```
}
```

```
u1 = 10
u2 = 10
```

7. **Borrowing and References**
   - Demonstrate &T and &mut T references and restrictions.

a) Borrowing

It is very inconvenient to pass the ownership of a variable to another function and then return the ownership. Rust supports a concept, borrowing, where the ownership of a value is transferred temporarily to an entity and then returned to the original owner entity.

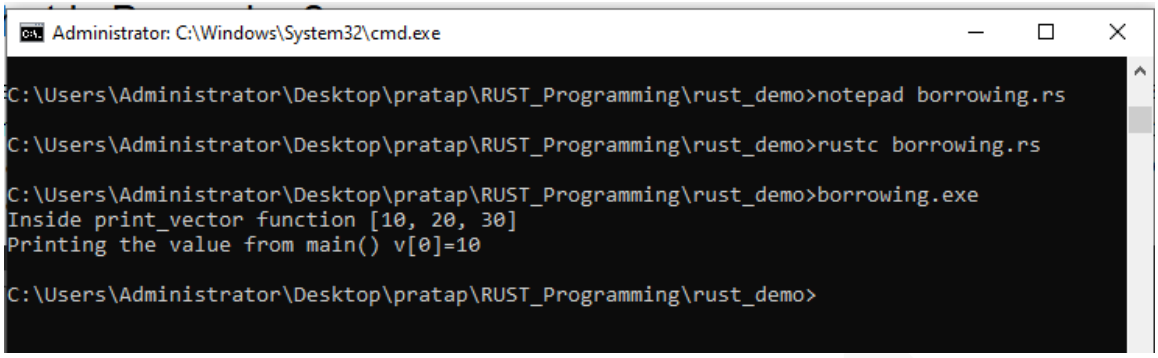
What is Borrowing?

When a function transfers its control over a variable/value to another function temporarily, for a while, it is called borrowing. This is achieved by passing a reference to the variable ***(& var_name)*** rather than passing the variable/value itself to the function. The ownership of the variable/ value is transferred to the original owner of the variable after the function to which the control was passed completes execution.

Program:

```
fn main(){
   // a list of nos
   let v = vec![10,20,30];
   print_vector(&v); // passing reference
   println!("Printing the value from main() v[0]={}",v[0]);
}
fn print_vector(x:&Vec<i32>){
   println!("Inside print_vector function {:?}",x);
}
```

Output:



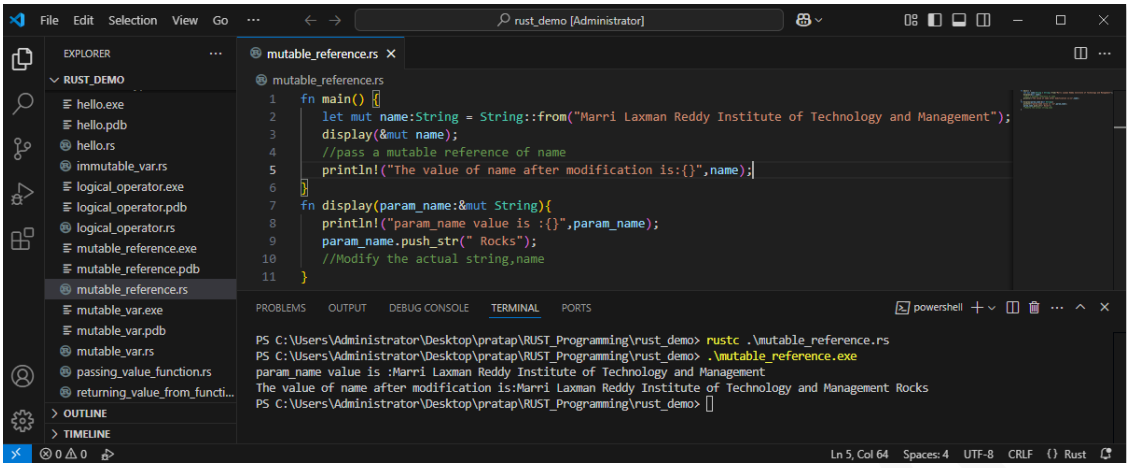
b) Mutable References

A function can modify a borrowed resource by using a *mutable reference* to such resource. A mutable reference is prefixed with ***&mut***. Mutable references can operate only on mutable variables.

Program:

```
fn main() {
   let mut name:String = String::from("Marri Laxman Reddy Institute of Technology and Management");
   display(&mut name);
   //pass a mutable reference of name
   println!("The value of name after modification is:{}",name);
}
fn display(param_name:&mut String){
   println!("param_name value is :{}",param_name);
   param_name.push_str(" Rocks");
   //Modify the actual string,name
}
```

Output:



8. **Lifetimes Example**
   - Use lifetime annotations in a function returning references.

A *lifetime* is a construct the compiler (or more specifically, its *borrow checker*) uses to ensure all borrows are valid. Specifically, a variable's lifetime begins when it is created and ends when it is destroyed. While lifetimes and scopes are often referred to together, they are not the same.

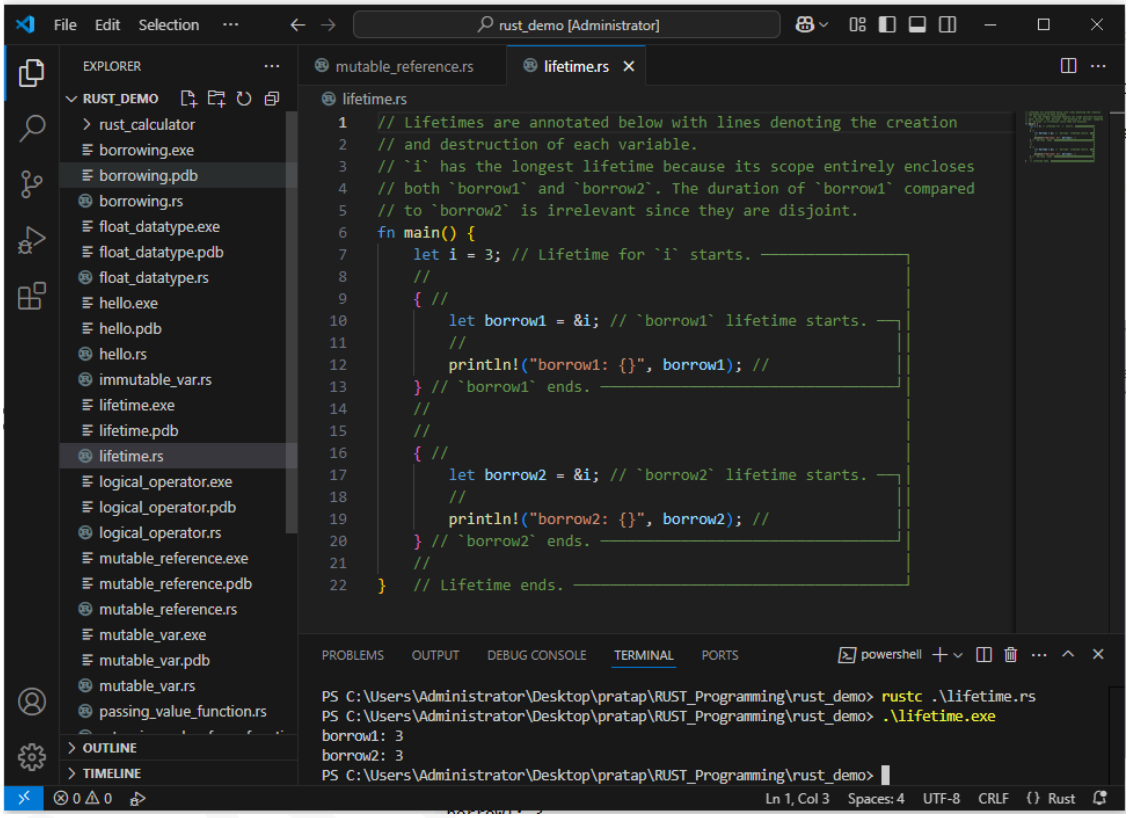
Take, for example, the case where we borrow a variable via &. The borrow has a lifetime that is determined by where it is declared. As a result, the borrow is valid as long as it ends before the lender is destroyed. However, the scope of the borrow is determined by where the reference is used.

In the following example and in the rest of this section, we will see how lifetimes relate to scopes, as well as how the two differ.

```
// Lifetimes are annotated below with lines denoting the creation
// and destruction of each variable.
// `i` has the longest lifetime because its scope entirely encloses
// both `borrow1` and `borrow2`. The duration of `borrow1` compared
// to `borrow2` is irrelevant since they are disjoint.
fn main() {
    let i = 3; // Lifetime for `i` starts. ────────────────────┐
    //                                                         │
    { //                                                        │
        let borrow1 = &i; // `borrow1` lifetime starts. ──┐    │
        //                                                │ │  │
        println!("borrow1: {}", borrow1); //              │ │  │
    } // `borrow1` ends. ──────────────────────────────────┘ │  │
    //                                                         │
    //                                                         │
```

```
    { //                                        |
        let borrow2 = &i; // `borrow2` lifetime starts. ──┐ |
        //                                        | |
        println!("borrow2: {}", borrow2); //          | |
    } // `borrow2` ends. ──────────────────────────────────┘ |
    //                                        |
}   // Lifetime ends. ──────────────────────────────────────┘
```

Output :



Note that no names or types are assigned to label lifetimes. This restricts how lifetimes will be able to be used as we will see

<u>Program2:</u>

```
// One input reference with lifetime `'a` which must live

// at least as long as the function.

fn print_one<'a>(x: &'a i32) {

    println!("`print_one`: x is {}", x);
```

```
}


// Mutable references are possible with lifetimes as well.
fn add_one<'a>(x: &'a mut i32) {
    *x += 1;
}


// Multiple elements with different lifetimes. In this case, it
// would be fine for both to have the same lifetime `'a`, but
// in more complex cases, different lifetimes may be required.
fn print_multi<'a, 'b>(x: &'a i32, y: &'b i32) {
    println!("`print_multi`: x is {}, y is {}", x, y);
}
// Returning references that have been passed in is acceptable.
// However, the correct lifetime must be returned.
fn pass_x<'a, 'b>(x: &'a i32, _: &'b i32) -> &'a i32 { x }
//fn invalid_output<'a>() -> &'a String { &String::from("foo") }
// The above is invalid: `'a` must live longer than the function.
// Here, `&String::from("foo")` would create a `String`, followed by a
// reference. Then the data is dropped upon exiting the scope, leaving
// a reference to invalid data to be returned.

fn main() {
    let x = 7;
    let y = 9;
```

```
    print_one(&x);
    print_multi(&x, &y);

    let z = pass_x(&x, &y);
    print_one(z);

    let mut t = 3;
    add_one(&mut t);
    print_one(&t);
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\lifetime_function_annotation.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\lifetime_function_annotation.exe
`print_one`: x is 7
`print_multi`: x is 7, y is 9
`print_one`: x is 7
`print_one`: x is 4
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

9. **If-Else Statement**
   - Even/odd checker or grade calculator.

<u>Program 1: even or odd</u>

```
fn main() {
  let num = 12;
  if num % 2==0 {
    println!("Even");
  } else {
    println!("Odd");
  }
}
```

<u>Output:</u>

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\even_odd.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\even_odd.exe
Even
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program 2: Grade calculator

```
use std::io;

fn main() {
    println!("Enter the number of subjects:");

    let mut input = String::new();
    io::stdin().read_line(&mut input).unwrap();
    let num_grades: usize = input.trim().parse().unwrap();

    let mut grades = Vec::new();

    for i in 0..num_grades {
        println!("Enter marks_subject {}:", i + 1);
        input.clear();
        io::stdin().read_line(&mut input).unwrap();
        let grade: f32 = input.trim().parse().unwrap();
        grades.push(grade);
    }

    let sum: f32 = grades.iter().sum();
    let average = sum / num_grades as f32;

    let grade_letter = if average >= 90.0 {
        "A"
    } else if average >= 80.0 {
        "B"
    } else if average >= 70.0 {
        "C"
    } else if average >= 60.0 {
        "D"
    } else {
        "F"
    };

    println!("Average Marks: {:.2}", average);
    println!("Calculated grade for marks is: {}", grade_letter);
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\grade_calculator.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\grade_calculator.exe
Enter the number of subjects:
5
Enter marks_subject 1:
80
Enter marks_subject 2:
70
Enter marks_subject 3:
60
Enter marks_subject 4:
50
Enter marks_subject 5:
90
Average Marks: 70.00
Calculated grade for marks is: C
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

10. **Looping Examples**
    - Use for, while, and loop to iterate over arrays or counters.

Rust provides different types of loops to handle looping requirements −

- while
- loop
- for

Definite Loop:

A loop the number of iterations of which is definite/fixed is termed as a definite loop.
The **for** loop is an implementation of a definite loop.

<u>Program 1: For Loop</u>

```
fn main(){
  let arr:[i32;4] = [10,20,30,40];
  println!("array is {:?}",arr);
  println!("array size is :{}",arr.len());

  for index in 0..4 {
    println!("index is: {} & value is : {}",index,arr[index]);
  }
}
```

<u>Output:</u>

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\for_loop.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\for_loop.exe
array is [10, 20, 30, 40]
array size is :4
index is: 0 & value is : 10
index is: 1 & value is : 20
index is: 2 & value is : 30
index is: 3 & value is : 40
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program 2: While loop

```
fn main(){
  let mut x = 0;
  while x < 10{
    x+=1;
    println!("inside loop x value is {}",x);
  }
  println!("outside loop x value is {}",x);
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\while.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\while.exe
inside loop x value is 1
inside loop x value is 2
inside loop x value is 3
inside loop x value is 4
inside loop x value is 5
inside loop x value is 6
inside loop x value is 7
inside loop x value is 8
inside loop x value is 9
inside loop x value is 10
outside loop x value is 10
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program: while loop using arrays:

```
fn main() {
   let numbers = [10, 20, 30, 40, 50];
   let mut index = 0;

   while index < numbers.len() {
      println!("Element at index {}: {}", index, numbers[index]);
      index += 1;
   }
```

}

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\while_loop_array.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\while_loop_array.exe
Element at index 0: 10
Element at index 1: 20
Element at index 2: 30
Element at index 3: 40
Element at index 4: 50
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program 3: Loop

```
fn main()
{
  //while true
  let mut x = 0;
  loop
{
     x+=1;
     println!("x={}",x);

     if x==15
 {
       break;
     }
  }
}
```

Output:

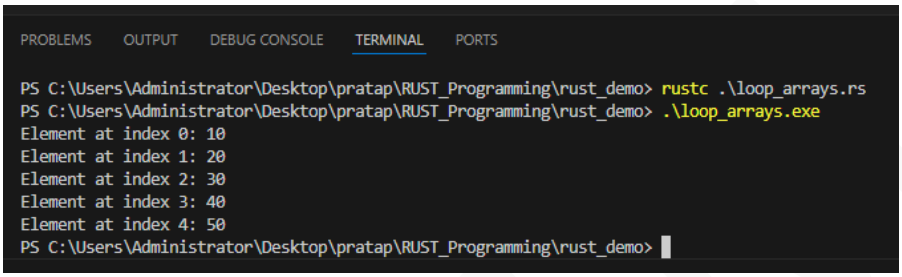
```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\loop.exe
x=1
x=2
x=3
x=4
x=5
x=6
x=7
x=8
x=9
x=10
x=11
x=12
x=13
x=14
x=15
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program loop_arrays:

```
fn main() {
   let numbers = [10, 20, 30, 40, 50];
   let mut index = 0;

   loop {
```

```
        if index >= numbers.len() {
            break; // Exit the loop when the end of the array is reached
        }

        println!("Element at index {}: {}", index, numbers[index]);
        index += 1; // Increment the index to move to the next element
    }
}
```

Output:



11. **Pattern Matching with match**
    - Match numbers to print weekdays or match enums.

Program:match.rs

```
use std::io;
fn main() {
    let mut input = String::new();
    println!("Enter a number (1-7) to get the corresponding weekday:");

    io::stdin().read_line(&mut input).unwrap();
    let day: i32 = input.trim().parse().unwrap();

    match day {
        1 => println!("Monday"),
        2 => println!("Tuesday"),
        3 => println!("Wednesday"),
        4 => println!("Thursday"),
        5 => println!("Friday"),
        6 => println!("Saturday"),
        7 => println!("Sunday"),
        _ => println!("Invalid day! Please enter a number between 1 and 7."),
    }
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

Element at index 3: 40
Element at index 4: 50
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc match.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> match.exe
Enter a number (1-7) to get the corresponding weekday:
5
Friday
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program: match_enums.rs

```
use std::io;
enum Weekday {
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday,
    Sunday,
}

fn main() {
    let mut input = String::new();
    println!("Enter the name of a weekday (e.g., Monday, Tuesday):");

    io::stdin().read_line(&mut input).unwrap();
    let input = input.trim();

    let today = match input {
        "Monday" => Weekday::Monday,
        "Tuesday" => Weekday::Tuesday,
        "Wednesday" => Weekday::Wednesday,
        "Thursday" => Weekday::Thursday,
        "Friday" => Weekday::Friday,
        "Saturday" => Weekday::Saturday,
        "Sunday" => Weekday::Sunday,
        _ => {
            println!("Invalid input! Please enter a valid weekday.");
            return;
        }
    };
```

```
    match today {
        Weekday::Monday => println!("Start of the work week!"),
        Weekday::Tuesday => println!("Second day of work."),
        Weekday::Wednesday => println!("Midweek!"),
        Weekday::Thursday => println!("Almost the weekend."),
        Weekday::Friday => println!("Weekend is here!"),
        Weekday::Saturday | Weekday::Sunday => println!("It's the weekend! Time to relax."),
    }
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\match_enums.exe
Enter the name of a weekday (e.g., Monday, Tuesday):
6
Invalid input! Please enter a valid weekday.
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\match_enums.exe
Enter the name of a weekday (e.g., Monday, Tuesday):
Sunday
It's the weekend! Time to relax.
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

12. **Nested Control Statements**
    - Combine loops and conditionals in one program (e.g., a simple number guessing game).

1. **Creating number guessing game project :**
   PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> cargo new number_guessing_game
   Creating binary (application) `number_guessing_game` package
   note: see more `Cargo.toml` keys and their definitions at https://doc.rust-lang.org/cargo/reference/manifest.html

2. **Go to project directory:**

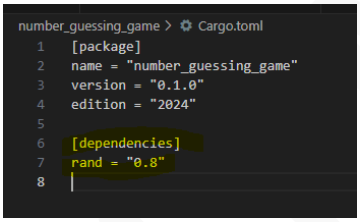
   PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> cd .\number_guessing_game\

3. **Go to main.rs file in the project directory and write the number guessing game code:**

```
use std::io;
use rand::Rng;

fn main() {
    let secret_number = rand::thread_rng().gen_range(1..=100);
    let mut attempts = 0;

    println!("Welcome to the Number Guessing Game!");
    println!("Try to guess the number between 1 and 100.");

    loop {
        println!("Enter your guess:");

        let mut guess = String::new();
        io::stdin().read_line(&mut guess).unwrap();

        let guess: i32 = match guess.trim().parse() {
            Ok(num) => num,
```

```
            Err(_) => {
                println!("Please enter a valid number.");
                continue;
            },
        };

        attempts += 1;

        if guess < secret_number {
            println!("Too low! Try again.");
        } else if guess > secret_number {
            println!("Too high! Try again.");
        } else {
            println!("Congratulations! You guessed the number in {} attempts.", attempts);
            break;
        }
    }
}
```

4. Adding dependencies to Cargo.toml



5. Build the Project

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\number_guessing_game> cargo build

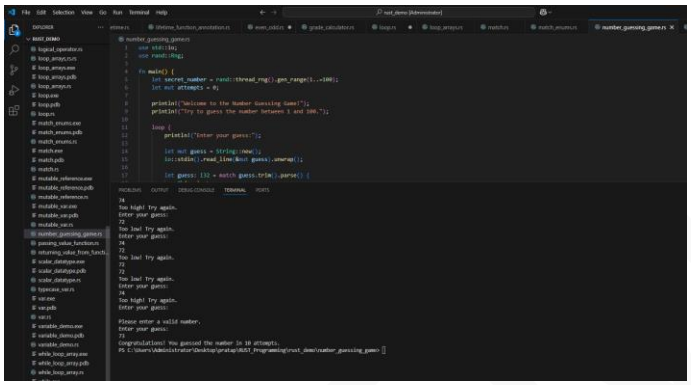
6. **Run the Project:**

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\number_guessing_game> cargo run

7. Output :

```
Welcome to the Number Guessing Game!
Try to guess the number between 1 and 100.
Enter your guess:
99
Too high! Try again.
Enter your guess:
1
Too low! Try again.
Enter your guess:
50
Too low! Try again.
Enter your guess:
60
Too low! Try again.
Enter your guess:
80
Too high! Try again.
Enter your guess:
70
Too low! Try again.
Enter your guess:
75
Too high! Try again.
Enter your guess:
72
Too low! Try again.
Enter your guess:
74
Too high! Try again.
```

```
Enter your guess:

Please enter a valid number.
Enter your guess:
73
Congratulations! You guessed the number in 10 attempts.
```



Module III – Functions and Structs

13. **Functions with Arguments and Return Values**
    - Create a function to calculate factorial or square of a number.

```
fn factorial(num: u32) -> u32 {
    if num == 0 || num == 1 {
        1
    } else {
        num * factorial(num - 1)
    }
}

fn square(num: i32) -> i32 {
    num * num
}

fn main() {
    let number = 5;

    let fact = factorial(number);
    println!("The factorial of {} is {}", number, fact);

    let sq = square(number as i32);
    println!("The square of {} is {}", number, sq);
}
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\factorial_square.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\factorial_square.exe
The factorial of 5 is 120
The square of 5 is 25
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

14. **Ownership in Functions**
    - Pass by value vs. reference in function parameters.

Program: Pass by value:

```
fn pass_by_value(x: i32) {
    println!("Received value: {}", x);
}

fn main() {
    let num = 10;
    pass_by_value(num); // Ownership transferred
    println!("Original value: {}", num); // `num` is still valid since `i32` implements Copy trait
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\pass_by_value.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\pass_by_value.exe
Received value: 10
Original value: 10
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program: Pass by reference:

```
fn pass_by_reference(x: &i32) {
    println!("Received reference to value: {}", x);
}

fn main() {
    let num = 10;
    pass_by_reference(&num); // Ownership remains with the caller
    println!("Original value: {}", num); // `num` is still valid
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\pass_by_reference.exe
Received reference to value: 10
Original value: 10
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

15. **Using Structs**
    - Define a Rectangle struct and calculate area.

Program:

```
struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    fn area(&self) -> f64 {
        self.width * self.height
    }
}

fn main() {
    let rect = Rectangle {
        width: 5.0,
        height: 10.0,
    };

    println!("Width: {}", rect.width);
    println!("Height: {}", rect.height);
    println!("Area of the rectangle: {}", rect.area());
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\struct_rectangle_area.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\struct_rectangle_area.exe
Width: 5
Height: 10
Area of the rectangle: 50
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

16. **Tuple Structs and Field Init Shorthand**
   - Define a Color(u8, u8, u8) and use field shorthand for init.

Program:

```
struct Color(u8, u8, u8);

fn main() {
    let red = 255;
    let green = 0;
    let blue = 128;

    // Use field init shorthand to initialize the Color tuple struct
    let my_color = Color(red, green, blue);

    println!("Color - Red: {}, Green: {}, Blue: {}", my_color.0, my_color.1, my_color.2);
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\struct_color.exe
Color - Red: 255, Green: 0, Blue: 128
```

17. **Enums and Pattern Matching**
   - Define an enum for TrafficLight and match its values.

Program:

```
enum TrafficLight {
    Red,
    Yellow,
    Green,
}

fn action(light: TrafficLight) {
    match light {
        TrafficLight::Red => println!("Stop!"),
        TrafficLight::Yellow => println!("Get ready to stop."),
        TrafficLight::Green => println!("Go!"),
    }
}

fn main() {
    let current_light = TrafficLight::Green;
    action(current_light);

    let next_light = TrafficLight::Yellow;
```

```
    action(next_light);

    let another_light = TrafficLight::Red;
    action(another_light);
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\trafficlight.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\trafficlight.exe
Go!
Get ready to stop.
Stop!
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

18. **Option Enum Usage**
    - Safe division function returning Option<f64>.

Program:

```
fn safe_divide(a: f64, b: f64) -> Option<f64> {
    if b == 0.0 {
        None // Return None if the denominator is zero
    } else {
        Some(a / b) // Return the result wrapped in Some if division is valid
    }
}

fn main() {
    let numerator = 10.0;
    let denominator = 2.0;

    match safe_divide(numerator, denominator) {
        Some(result) => println!("Result: {}", result),
        None => println!("Error: Division by zero is not allowed."),
    }

    let denominator_zero = 0.0;

    match safe_divide(numerator, denominator_zero) {
        Some(result) => println!("Result: {}", result),
        None => println!("Error: Division by zero is not allowed."),
    }
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\safe_division.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\safe_division.exe
Result: 5
Error: Division by zero is not allowed.
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Module IV – Project Management and Smart Pointers

19. **Error Handling with Result**
    - File reading or division with error handling using Result.

Program: program with error: error_handling.rs

```
use std::fs::File;
use std::io::{self, Read};

fn read_file_content(file_path: &str) -> Result<String, io::Error> {
    let mut file = File::open(file_path)?;
    let mut content = String::new();
    file.read_to_string(&mut content)?;
    Ok(content)
}

fn main() {
    let file_path = "example.txt"; // Provide the path to your file here

    match read_file_content(file_path) {
        Ok(content) => println!("File content:\n{}", content),
        Err(e) => println!("Error reading file: {}", e),
    }
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\error_handling.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\error_handling.exe
Error reading file: The system cannot find the file specified. (os error 2)
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Program: program for error handling:

```
fn safe_divide(a: f64, b: f64) -> Result<f64, &'static str> {
    if b == 0.0 {
```

```
        Err("Division by zero is not allowed")
    } else {
        Ok(a / b)
    }
}

fn main() {
    let numerator = 10.0;
    let denominator = 2.0;

    match safe_divide(numerator, denominator) {
        Ok(result) => println!("Result: {}", result),
        Err(e) => println!("Error: {}", e),
    }

    let denominator_zero = 0.0;

    match safe_divide(numerator, denominator_zero) {
        Ok(result) => println!("Result: {}", result),
        Err(e) => println!("Error: {}", e),
    }
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc safe_division_with_error_handling.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> safe_division_with_error_handling.exe
Result: 5
Error: Division by zero is not allowed
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

20. **Using Box, Rc, and RefCell**
    - Demonstrate smart pointers with a simple linked list or counter program.

Program:

```
use std::rc::Rc;
use std::cell::RefCell;

#[derive(Debug)]
struct Node {
    value: i32,
    next: Option<Rc<RefCell<Node>>>, // Use Rc for shared ownership and RefCell for mutability
}

fn main() {
    // Create nodes using Rc and RefCell
```

```
    let first = Rc::new(RefCell::new(Node {
        value: 1,
        next: None,
    }));

    let second = Rc::new(RefCell::new(Node {
        value: 2,
        next: None,
    }));

    let third = Rc::new(RefCell::new(Node {
        value: 3,
        next: None,
    }));

    // Link the nodes to form the list
    first.borrow_mut().next = Some(second.clone());
    second.borrow_mut().next = Some(third.clone());

    // Traverse and print the linked list
    let mut current = Some(first.clone());

    println!("Linked List:");
    while let Some(node) = current {
        println!("{}", node.borrow().value);
        current = node.borrow().next.clone(); // Move to the next node
    }
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\pointer_linkedlist.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\pointer_linkedlist.exe
Linked List:
1
2
3
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

Module- V – Object oriented Programming

21. **"Polymorphic Behavior with Trait Objects: A Speakable Animal Zoo"**
    Covers: Traits, dynamic dispatch, trait objects (&dyn Trait)

Program:speakable_zoo.rs

```
// Define the Speakable trait
```

```
trait Speakable {
    fn speak(&self) -> String;
}

// Implement the Speakable trait for Dog
struct Dog;

impl Speakable for Dog {
    fn speak(&self) -> String {
        String::from("Woof! I'm a dog.")
    }
}

// Implement the Speakable trait for Cat
struct Cat;

impl Speakable for Cat {
    fn speak(&self) -> String {
        String::from("Meow! I'm a cat.")
    }
}

// Implement the Speakable trait for Bird
struct Bird;

impl Speakable for Bird {
    fn speak(&self) -> String {
        String::from("Chirp! I'm a bird.")
    }
}

// A Zoo that holds Speakable animals
struct Zoo {
    animals: Vec<Box<dyn Speakable>>, // Use trait objects for polymorphic behavior
}

impl Zoo {
    fn new() -> Self {
        Zoo { animals: Vec::new() }
    }

    fn add_animal(&mut self, animal: Box<dyn Speakable>) {
        self.animals.push(animal);
    }

    fn make_all_speak(&self) {
        for animal in &self.animals {
            println!("{}", animal.speak());
```

```
        }
    }
}

fn main() {
    let mut zoo = Zoo::new();

    // Add different types of animals to the zoo
    zoo.add_animal(Box::new(Dog));
    zoo.add_animal(Box::new(Cat));
    zoo.add_animal(Box::new(Bird));

    // Make all animals speak
    println!("Welcome to the Speakable Animal Zoo!");
    zoo.make_all_speak();
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\speakable_zoo.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\speakable_zoo.exe
Welcome to the Speakable Animal Zoo!
Woof! I'm a dog.
Meow! I'm a cat.
Chirp! I'm a bird.
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

22. **"Implementing Strategy Pattern with Traits for Payment Processing"**
    Covers: Object-oriented design using traits and Box<dyn Trait> for dynamic strategy switching

```
Program: payment_processing.rs
// Define a trait for payment strategies
trait PaymentStrategy {
    fn pay(&self, amount: f64);
}

// Implement a payment strategy for Credit Card
struct CreditCardPayment;

impl PaymentStrategy for CreditCardPayment {
    fn pay(&self, amount: f64) {
        println!("Processing credit card payment of ${:.2}", amount);
    }
}
```

```
// Implement a payment strategy for PayPal
struct PayPalPayment;

impl PaymentStrategy for PayPalPayment {
    fn pay(&self, amount: f64) {
        println!("Processing PayPal payment of ${:.2}", amount);
    }
}

// Implement a payment strategy for Bitcoin
struct BitcoinPayment;

impl PaymentStrategy for BitcoinPayment {
    fn pay(&self, amount: f64) {
        println!("Processing Bitcoin payment of ${:.2}", amount);
    }
}

// Payment context to switch strategies dynamically
struct PaymentProcessor {
    strategy: Box<dyn PaymentStrategy>, // Trait object for dynamic dispatch
}

impl PaymentProcessor {
    // Create a new payment processor with a given strategy
    fn new(strategy: Box<dyn PaymentStrategy>) -> Self {
        PaymentProcessor { strategy }
    }

    // Change the payment strategy
    fn set_strategy(&mut self, strategy: Box<dyn PaymentStrategy>) {
        self.strategy = strategy;
    }

    // Process payment using the current strategy
    fn process_payment(&self, amount: f64) {
        self.strategy.pay(amount);
    }
}

fn main() {
    // Initialize payment processor with Credit Card strategy
    let mut processor = PaymentProcessor::new(Box::new(CreditCardPayment));
    processor.process_payment(100.0);

    // Switch to PayPal strategy
    processor.set_strategy(Box::new(PayPalPayment));
```

```
    processor.process_payment(200.0);

    // Switch to Bitcoin strategy
    processor.set_strategy(Box::new(BitcoinPayment));
    processor.process_payment(300.0);
}
```

Output:

```
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> rustc .\payment_processing.rs
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo> .\payment_processing.exe
Processing credit card payment of $100.00
Processing PayPal payment of $200.00
Processing Bitcoin payment of $300.00
PS C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo>
```

23. **"Trait-Based Drawing Application with Heterogeneous UI Components"**
Covers: Object-oriented characteristics, trait objects, and allowing different drawable types in a single collection

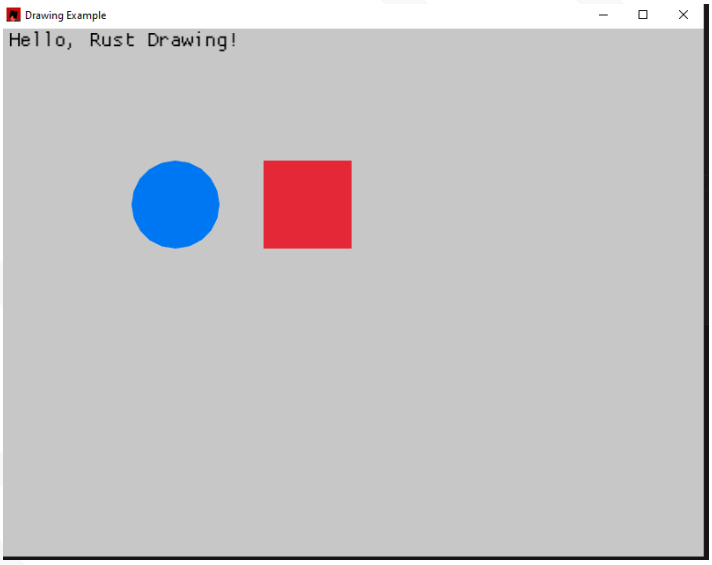
Program:

```
use macroquad::prelude::*;

#[macroquad::main("Drawing Example")]
async fn main() {
    loop {
        // Clear the screen with a background color
        clear_background(LIGHTGRAY);

        // Draw a circle
        draw_circle(200.0, 200.0, 50.0, BLUE);

        // Draw a rectangle
        draw_rectangle(300.0, 150.0, 100.0, 100.0, RED);

        // Draw text
        draw_text("Hello, Rust Drawing!", 10.0, 20.0, 30.0, BLACK);

        // End the drawing frame
        next_frame().await;
    }
}
```

Dependencies:

drawing_application > Cargo.toml

```toml
[package]
name = "drawing_application"
version = "0.1.0"
edition = "2024"

[dependencies]
macroquad = "0.3"
```

Output:

```
PS
C:\Users\Administrator\Desktop\pratap\RUST_Programming\rust_demo\drawing_application>
cargo run
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.04s
    Running `target\debug\drawing_application.exe`
WGL: failed to create 3.2 context, trying 2.1
GL_VERSION: 4.6.0 - Build 32.0.101.6078
```